

Der Agent ist kein Chat, sondern ein neuer Mitarbeiter: Kontext, Werkzeuge, Skills richtig onboarden

Worauf das aufbaut: Diese Unterlage ist eine eigene Aufbereitung und Einordnung des Videos von **Open Residency Podcast**. Die Gedanken stammen von dort, Auswahl, Gliederung, Beispiele und Kommentierung sind meine. Wörtliche Zitate sind als solche gekennzeichnet, Übersetzungen aus dem Englischen sind eigene. Originalquelle: <https://www.youtube.com/watch?v=5p-sq8v3OXw>

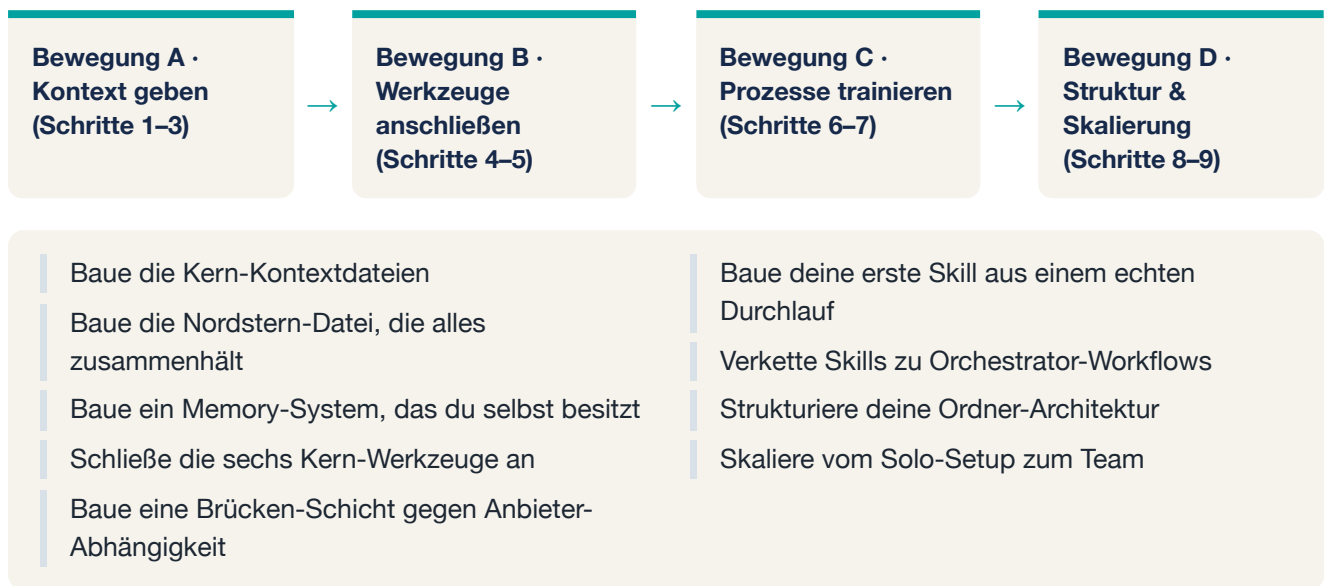
Kern-These

Chat ist Frage zu Antwort: du fragst etwas, bekommst eine Antwort und musst die eigentliche Arbeit danach selbst erledigen. Ein Agent ist Ziel zu Ergebnis: du gibst ein Ziel vor, er zerlegt es in einen Plan und arbeitet eine Schleife aus Beobachten, Denken und Handeln ab, bis ein fertiges Ergebnis vor dir liegt. Remys Kernpunkt: ein roher Agent direkt aus der Box ist trotzdem meistens enttäuschend, egal wie stark das dahinterliegende Modell ist. Er kennt dich nicht, hat keine Werkzeuge und weiß nicht, wie du Dinge erledigt haben willst. Genau das behebst du, indem du ihn onboardest wie einen echten neuen Mitarbeiter: über Kontext, Werkzeuge und Skills. Das Modell selbst ist dabei nur der Motor, nicht der Hebel mit der größten Wirkung.

Flip: Chat optimiert die einzelne Frage. Ein Agent übernimmt den ganzen Vorgang, aber nur, wenn er vorher onboardet wurde wie ein Mitarbeiter, nicht wie ein Suchfeld.

Die Größenordnung (Remys Eigenangaben, ungeprüft, keine unabhängige Quelle): ein Team-Mitglied mit reinem Chat-Einsatz bleibt bei etwa der eigenen Kapazität. Mit einem rohen, ungetunten Agenten nennt Remy den Mittelwert der Spanne „5 bis 10x“ produktiver, also etwa das Siebenfache. Mit einem vollständig onboardeten Agenten (Kontext, Werkzeuge, Skills sauber aufgesetzt) nennt er die Zielgröße „100x employee“. Zwischen roh und onboardet liegt also nach seiner eigenen Einschätzung der größere Sprung, nicht zwischen Chat und Agent.

Landkarte: 9 Schritte, 4 Bewegungen



Wo stehst du gerade? (Einstiegs-Weiche)

- „Ich frage meinen Agenten jedes Mal wieder dasselbe, weil er sich an nichts erinnert.“ Engpass: fehlender Kontext-Unterbau → Schritt 1.
- „Mein Agent kann reden, aber nichts wirklich tun.“ Engpass: keine angeschlossenen Werkzeuge → Schritt 4.
- „Ich erkläre jedes Mal aufs Neue, wie ich das Ergebnis will.“ Engpass: keine wiederverwendbare Skill → Schritt 6.
- „Ich habe zehn Ordner mit eigenen Agenten-Namen und blicke selbst nicht mehr durch.“ Engpass: Ordner-Architektur ohne System → Schritt 8.

Schritt 1 · Baue die Kern-Kontextdateien (A)

Context ist alles, was der Agent bereits wissen sollte, bevor du ihm eine Aufgabe gibst. Für Remy heißt das konkret: eine Datei über ihn selbst als Person, eine über die Geschäftsinformationen, eine über das ideale Kundenprofil, eine über den Angebotskatalog samt Preisen und Upsell-Logik. Jede davon liegt als eigene Markdown-Datei vor, nicht als PDF oder Word-Dokument, weil ein Agent Markdown ohne Umwege einlesen kann, während er ein PDF erst mühsam entpacken muss. Gebaut werden diese Dateien am einfachsten im Dialog: eine neue Chat-Sitzung öffnen, den Agenten bitten, dich zu interviewen, und die Antworten per Diktat einsprechen, statt sie zu tippen.

O-Ton:

„Context is literally like everything that the agent should already know before you give it a task.“ Und zur Methode: „I said to Claude, interview me, like to extract the information from my brain to build these files, and I sat there using Whisper Flow answering all the questions for like 3 hours.“

Werkzeug · Das Kontext-Dateien-Set:

- (1) about.md (wer du bist, deine Geschichte).
- (2) business-info.md (was das Geschäft macht).
- (3) ideal-customer-profile.md (wen du bedienst).
- (4) offer-catalog.md (Angebote, Preise, Verknüpfungen).
- (5) Bauen per Interview-Methode: neue Sitzung, „interviewe mich“, Antworten per Diktat.

Falle & Fix: Falle, die vom Agenten vorgeschlagenen Inhalte ungeprüft übernehmen, etwa ein falsches Problem im Kundenprofil, das sich danach durch die ganze Marketing-Copy zieht.

Fix, jede Zeile selbst lesen, bevor sie freigegeben wird. Diese Dateien sind Firmen-Assets, kein Wegwerf-Text.

Reflexionsfragen: Welche deiner Kontextdateien hast du zuletzt wirklich Zeile für Zeile gegengelesen, statt sie durchzuwinken? Gibt es eine Kontext-Lücke bei dir, die ein neuer Mitarbeiter sofort bemerken würde, dein Agent aber nie gefragt hat?

Schritt 2 · Baue die Nordstern-Datei, die alles zusammenhält (A)

Jede neue Sitzung mit einem Agenten startet bei null, es sei denn, eine bestimmte Datei sagt ihm sofort, wer du bist. Diese Datei heißt je nach Anbieter `claude.md` oder `agents.md` und wird automatisch in jede neue Sitzung geladen, ohne dass du sie erwähnen musst. Remy demonstriert das live: ein leerer Ordner ohne diese Datei, und der Agent weiß nicht einmal, wer ihn gerade fragt. Sobald die Datei da ist und auf den Kontext-Ordner mit den übrigen Dateien verweist, kennt er sofort Name, Geschäft, Werte und Vorlieben.

O-Ton:

„Any markdown file in that folder titled `claude.md` gets automatically loaded in. So your `claude.md` or `agents.md`, that's your north star markdown file.“ Und im Text der Datei selbst: „Assumptions are the enemy. If the answer isn't there, ask me.“

Werkzeug · Die Nordstern-Datei:

- (1) Kurzprofil an den Anfang (wer du bist, was das Geschäft macht).
- (2) Expliziter Verweis auf den Kontext-Ordner („lies das vor jeder Aufgabe“).
- (3) Eine Regel gegen Rateverhalten aufnehmen: fehlt die Information, nachfragen statt annehmen.
- (4) Mit dem @-Symbol referenzierte Dateien werden direkt geladen statt erst gesucht, das spart einen Schleifendurchlauf.

Falle & Fix: Falle, den Kontext nur lose in Unterordnern liegen zu lassen, ohne dass die Nordstern-Datei explizit darauf zeigt. Dann muss der Agent erst raten und suchen, bevor er überhaupt anfangen kann, und die erste Antwort ist entsprechend dünn.

Fix, die Nordstern-Datei immer als ersten Handgriff eines neuen Projekts anlegen und den Verweis auf den Kontext-Ordner hineinschreiben, bevor irgendetwas anderes passiert.

Reflexionsfragen: Wann hast du zuletzt eine neue Sitzung gestartet, in der du dem Agenten erst mühsam wieder erklären musstest, wer du bist? Zeigt deine Nordstern-Datei aktuell auf alles, was wirklich gebraucht wird, oder ist sie schon wieder länger, als sie sein müsste?

Schritt 3 · Baue ein Memory-System, das du selbst besitzt (A)

Kontextdateien geben dem Agenten das Ausgangswissen, aber sie wachsen nicht von selbst mit jeder Korrektur mit. Dafür braucht es eine eigene `memory.md`: eine Datei, die gelernte Regeln und Präferenzen sammelt, jedes Mal, wenn du etwas korrigierst. Remy zeigt das live an einer einfachen Regel („Websites immer im Dunkelmodus bauen, nie im Hellmodus“), die sofort in die Memory-Datei geschrieben wird und ab dann in jeder künftigen Sitzung gilt. Viele Anbieter haben inzwischen ein eingebautes Memory-Feature nachgerüstet, aber das bleibt eine Blackbox: du siehst nicht, was gespeichert wurde, und es gehört dem Anbieter, nicht dir.

O-Ton:

„When I correct you, write lessons into this memory file as a new rule and then check it before every task. This is how you go from good assistant to know exactly how I like things done.“ Zum eingebauten Anbieter-Speicher: „It's still almost a little bit of a black box. Claude kind of owns it.“

Werkzeug · Das Memory-Protokoll:

- (1) Eigene memory.md anlegen, in der Nordstern-Datei referenzieren.
- (2) Instruktion hineinschreiben: bei jeder Korrektur eine neue Regel dort ablegen.
- (3) Vor jeder Aufgabe die Memory-Datei lesen lassen.
- (4) Etwa alle sechs Monate eine Aufräum-Runde einlegen: „prüfe meine Memory-Regeln und entferne, was sich widerspricht.“

Falle & Fix: Falle, sich allein auf das eingebaute Memory-Feature des Anbieters zu verlassen. Das funktioniert nur innerhalb dieses einen Werkzeugs und wandert nicht mit, wenn du zu einem anderen Harness wechselst.

Fix, eine eigene, portable memory.md führen, die dir gehört und die du in jedes neue Werkzeug einfach mitnimmst.

Reflexionsfragen: *Wo hast du zuletzt eine Korrektur nur mündlich gegeben, statt sie irgendwo dauerhaft festzuhalten? Verlässt du dich bei irgendeinem Werkzeug gerade auf ein eingebautes Gedächtnis, das dir nicht wirklich gehört?*

Schritt 4 · Schließe die sechs Kern-Werkzeuge an (B)

Kontext allein macht den Agenten nicht handlungsfähig, er braucht Werkzeuge, um tatsächlich etwas zu tun. Das läuft über MCP (Model Context Protocol), eine Art Übersetzer, der zwischen dem Agenten und externen Werkzeugen vermittelt, ohne dass dafür eigene, teure Entwicklungsarbeit nötig wäre. Remy nennt sechs Werkzeuge, die praktisch jeder anschließen sollte: Apify (eine Bibliothek fertiger Scraper für Instagram, YouTube, LinkedIn und mehr), Firecrawl (liest ganze Webseiten statt nur zu suchen, nützlich für CRO-Audits oder Konkurrenz-Analysen), Chrome DevTools und Playwright (geben dem Agenten einen echten Browser, mit dem er Formulare ausfüllen oder eigene Arbeit gegenprüfen kann), sowie Higgsfield (Bild- und Videogenerierung, weil Claude von Haus aus keine Bilder erzeugen kann).

O-Ton:

„MCPs are just like this translator that sits in the middle between Claude and your tools.“ Zur Sicherheitsabwägung: „I would say if you're really conscious about it, give it these tools but give it what's called read-only access.“

Werkzeug · Die Sechs-Werkzeuge-Checkliste:

- (1) Apify für Scraping-Aufgaben.
- (2) Firecrawl fürs echte Lesen ganzer Seiten.
- (3) Chrome DevTools zum Gegenprüfen eigener Arbeit im Browser.
- (4) Playwright, wenn der bereits eingeloggte Browser gebraucht wird.
- (5) Higgsfield für Bild- und Videoerzeugung.
- (6) Bei jedem neuen Werkzeug zuerst prüfen, ob es eine eigene MCP-Verbindung anbietet, bevor eine Alternative ohne diese Anbindung gewählt wird.

Falle & Fix: Falle, aus Bequemlichkeit sofort alles auf „always allow“ zu stellen, ohne den Trade-off bewusst abzuwägen.

Fix, bei sensiblen Aktionen (Zugriff, Geld, Versand) erst mit reinem Lesezugriff starten und die Freigabe erst hochschrauben, wenn sich Vertrauen aufgebaut hat.

Reflexionsfragen: Welches deiner angeschlossenen Werkzeuge hat mehr Freigabe, als du dir bei genauerem Hinsehen eigentlich wohl dabei fühlst? Gibt es ein Werkzeug in deinem Alltag, das noch keine MCP-Anbindung hat und deshalb Umwege über die Weboberfläche erzwingt?

Schritt 5 · Baue eine Brücken-Schicht gegen Anbieter-Abhängigkeit (B)

Remy verbindet seine Werkzeuge ursprünglich alle direkt in einem einzigen Anbieter. Dann fällt dieser Anbieter für eine Stunde komplett aus, mitten in einer Arbeitsphase, und er kann nirgendwo weiterarbeiten, weil alle fünfzig Werkzeuge nur dort angeschlossen sind. Seine Lösung ist Composio: eine Zwischenschicht, die selbst wie ein Werkzeug-Marktplatz funktioniert, aber sich nur einmal mit jedem echten Werkzeug verbindet und diese Verbindung dann an jeden beliebigen Agenten-Harness weiterreicht. Ein Ausfall bei einem Anbieter bedeutet dann nicht mehr, dass die eigene Werkzeug-Infrastruktur mit ausfällt.

O-Ton:

„What Composio does, is like an MCP for MCPs. It's like a bridge basically.“ Zur Motivation: „Claude went down for an hour the other week. Like the servers were down and I had so much work to do. I had to open up Codex to work in there, but I started to get to Codex and it didn't have any of my tools.“

Werkzeug · Die Brücken-Prüfung:

- (1) Prüfen, wie viele Werkzeuge direkt nur in einem einzigen Anbieter verbunden sind.
- (2) Bei mehr als einer Handvoll eine Bridge-Schicht wie Composio dazwischenschalten.
- (3) Jedes Werkzeug nur einmal dort verbinden, dann an jeden Harness weiterreichen.
- (4) Mehrfach-Konten (etwa mehrere Gmail-Adressen), die ein einzelner Anbieter nicht sauber trennt, über die Bridge lösen.

Falle & Fix: Falle, fünfzig Werkzeuge einzeln in genau einem Anbieter zu verbinden und damit unbemerkt einen einzigen Ausfallpunkt für die gesamte eigene Arbeitsfähigkeit zu bauen.

Fix, eine Bridge-Schicht einmal aufsetzen, dann ist ein Anbieter-Ausfall ein Umweg, kein Stillstand.

Reflexionsfragen: *Wie viele deiner Werkzeuge hängen aktuell an genau einem Anbieter, ohne dass du dir dessen bewusst warst? Was würde bei dir tatsächlich stillstehen, wenn dein Haupt-Agenten-Anbieter morgen eine Stunde ausfällt?*

Schritt 6 · Baue deine erste Skill aus einem echten Durchlauf (c)

Eine Skill ist die Standardarbeitsanweisung für den Agenten, so wie man sie früher einer virtuellen Assistenz gegeben hätte, nur als Markdown-Datei statt als Google-Doc. Remy unterscheidet zwei Bauwege: goal-first (von Anfang an sagen „bau mir eine Skill dafür“) und process-first (erst einen echten Fall komplett durcharbeiten, mit allen Korrekturen, und danach sagen „mach daraus eine Skill“). Neunzig Prozent seiner eigenen Skills entstehen process-first, weil der Agent dann aus einem echten, bereits korrigierten Durchlauf lernt statt aus einer abstrakten Vorgabe. Wichtig dabei: die erste Version einer Skill ist praktisch nie perfekt, die meisten seiner Skills laufen inzwischen in Version fünf bis zehn.

O-Ton:

„The process first way is the way I build 90% of my skills.“ Und zur Reife einer Skill: „Most of my skills that I use are on like V5 to 10. The first time you build them, the V1 of the skill is never going to be perfect.“

Werkzeug · Die Prozess-zuerst-Methode:

- (1) Eine reale Aufgabe einmal komplett mit dem Agenten durchgehen, inklusive aller Korrekturen unterwegs.
- (2) Am Ende sagen: „mach aus diesem Prozess eine Skill.“
- (3) Die eingebaute Skill-Creator-Skill übernimmt das Verpacken automatisch.
- (4) Nach jedem weiteren Einsatz nachschärfen: „mir gefällt an X nicht, ändere das“, statt die Skill unangetastet zu lassen.

Falle & Fix: Falle, bei einem noch unbekannten, komplexen Prozess sofort goal-first eine Skill zu verlangen, ohne den Prozess vorher einmal wirklich durchlebt zu haben. Das Ergebnis ist eine dünne Erst-Version ohne die feinen Korrekturen, die erst durch echte Arbeit sichtbar werden.

Fix, bei allem, was du zum ersten Mal machst, zuerst den echten Durchlauf fahren und erst danach die Skill daraus bauen lassen.

Reflexionsfragen: Welche Aufgabe hast du in den letzten Wochen zwei- oder dreimal fast identisch erledigt, ohne daraus eine Skill zu machen? Welche deiner bestehenden Skills steckt noch in Version eins fest, obwohl du sie seither mehrfach genutzt hast?

Schritt 7 · Verkette Skills zu Orchestrator-Workflows (C)

Sobald mehrere Einzel-Skills existieren, lohnt sich eine weitere Ebene: eine Orchestrator-Skill, die selbst keine eigene Arbeit erledigt, sondern nur festlegt, in welcher Reihenfolge andere Skills aufgerufen werden. Remys Beispiel ist sein YouTube-Publish-Workflow: die Skill selbst enthält nur den Hinweis, zuerst die Titel-Skill zu benutzen, dann die Thumbnail-Skill, dann die nächste, in genau dieser Reihenfolge. Die eigentliche Fachkompetenz, welche Reihenfolge sinnvoll ist (etwa: erst der Titel, dann das dazu passende Thumbnail), bleibt dabei eine strategische Entscheidung, die der Mensch trifft und die Skill nur festhält.

O-Ton:

„You can see you've got like almost an orchestrator skill that will call the other skills in sequence.“ Zum Wert der Reihenfolge selbst: „Some people want to look at thumbnails before they do titles, and without giving it that input, it would not know that.“

Werkzeug · Der Orchestrator-Aufbau:

- (1) Einzel-Skills schlank und auf eine Aufgabe fokussiert halten.
- (2) Eine übergeordnete Skill anlegen, die nur die Reihenfolge nennt, in der andere Skills aufgerufen werden.
- (3) Die fachliche Reihenfolge-Entscheidung selbst treffen, nicht dem Agenten überlassen.
- (4) Bei wiederkehrenden Mehrschritt-Prozessen (Content-Veröffentlichung, Kampagnen-Setup) zuerst prüfen, ob eine Verkettung bestehender Skills reicht, bevor eine komplett neue Skill entsteht.

Falle & Fix: Falle, eine einzige riesige Skill zu bauen, die alles auf einmal abdeckt, wodurch sie unübersichtlich und schwer wartbar wird.

Fix, mehrere kleine, klar abgegrenzte Skills bauen und sie erst über eine schlanke Orchestrator-Skill verketteten.

Reflexionsfragen: Welcher deiner mehrstufigen, wiederkehrenden Prozesse besteht eigentlich schon aus Einzel-Skills, die nur noch nie in eine feste Reihenfolge gebracht wurden? Gibt es eine deiner Skills, die inzwischen zu viel auf einmal macht und in mehrere aufgeteilt gehört?

Schritt 8 · Strukturiere deine Ordner-Architektur (D)

Remy hat mit sehr granularen, nach Rollen benannten Ordnern angefangen (fast wie einzelne „Mitarbeiter“ mit eigenem Namen) und ist davon bewusst abgerückt. Sein heutiges Bild ist eine Holding-Gesellschaft: ein zentraler OS-Ordner ganz oben, darunter ein Ordner je Geschäftsbereich oder Lebensbereich (Firma A, Firma B, Gesundheit), dazu ein loser „aktiv“-Ordner für Einzelprojekte, die noch keinem Bereich zugeordnet sind. Darüber liegt eine zweite Unterscheidung: globaler Kontext (Nordstern-Datei auf OS-Ebene, gilt überall) versus projektspezifischer Kontext (eigene Nordstern-Datei in einem Unterordner, die sich mit der globalen zusammen aufbaut, nicht sie ersetzt).

O-Ton:

„The more I build and work, it's such an annoying, inefficient way to have things... I just run everything out of this OS folder.“ Zur zweistufigen Ebene: „You have context, tools and skills, and you can have them at a global versus a project level. The easiest way to think of global is like a holding company, and then the project level is like the sub companies.“

Werkzeug · Der Ordner-Bauplan:

- (1) Einen zentralen OS-Ordner anlegen.
- (2) Darunter je Geschäfts- oder Lebensbereich einen eigenen Ordner.
- (3) Einen „aktiv“-Ordner für lose Einzelprojekte ohne festen Platz.
- (4) Eine globale Nordstern-Datei auf OS-Ebene, zusätzliche projektspezifische Nordstern-Dateien in Unterordnern, die sich stapeln statt zu ersetzen.

Falle & Fix: Falle, für jede Nische einen eigenen, mit Namen versehenen „Agenten“-Ordner zu bauen (Remys eigenes Beispiel: ein Marketing-Agent namens Marty), was schnell in unnötiger Komplexität endet.

Fix, einen einzigen Ordnerbaum führen und Spezialisierung über Skills lösen, nicht über separat benannte Persona-Agenten.

Reflexionsfragen: Hast du irgendwo einen zu granularen, nach Rolle benannten Ordner, der eigentlich nur eine Skill innerhalb einer größeren Struktur sein müsste? Wo fehlt dir aktuell eine klare globale Nordstern-Datei, weil der Kontext nur lokal in einem einzelnen Projekt liegt?

Schritt 9 · Skaliere vom Solo-Setup zum Team (D)

Sobald ein ganzes Unternehmen KI-nativ werden soll, gibt es laut Remy zwei Wege. Top-down bedeutet: externe KI-Spezialisten interviewen jede Abteilung und bauen die Automatisierung stellvertretend. Das Problem dabei ist, dass diese Spezialisten nie so tief in jeder einzelnen Rolle stecken können wie die Person, die sie täglich ausübt, wodurch dieser Weg entweder unmöglich viele Spezialisten braucht oder oberflächlich bleibt. Bottom-up bedeutet: jeder Mitarbeiter wird selbst KI-kompetent gemacht und baut sein eigenes kleines Betriebssystem, genau wie Remy es für sich selbst zeigt. Damit einher geht sofort eine ungelöste Eigentumsfrage: gehört eine selbstgebaute Skill dem Mitarbeiter oder dem Unternehmen, wenn die Person das Unternehmen verlässt?

O-Ton:

„I think the best way to get your company AI native is to get your employees trained up to be AI literate, get them building their own personal operating system... and then you automate it from the bottom up.“ Zur offenen Eigentumsfrage: „If they're assets for the company, are you going to just let them walk away with the skills when they leave, or do you as a company own them? It's a difficult one, I can't say.“

Werkzeug · Die Skalierungs-Reihenfolge:

- (1) Jede Person zuerst ihr eigenes kleines Betriebssystem bauen lassen (Kontext, Werkzeuge, Skills für die eigene Rolle).
- (2) Anreize setzen, die das Bauen sichtbar machen (Skill-des-Monats, Build-Wochen mit Preis).
- (3) Regelmäßig einen Austausch-Termin einplanen, an dem Mitarbeitende zeigen, was sie gebaut haben.
- (4) Die Eigentumsfrage bei Skills, die faktisch Firmenwissen enthalten, früh und bewusst klären, statt sie erst beim ersten Weggang zu entdecken.

Falle & Fix: Falle, top-down zu versuchen und externe KI-Spezialisten stellvertretend für jede Rolle bauen zu lassen, was entweder ein Spezialist pro Mitarbeiter erfordert oder oberflächlich bleibt.

Fix, bottom-up starten: jeder automatisiert zuerst den eigenen Job, das Unternehmen wächst danach in die Breite.

Reflexionsfragen: Wenn du gerade ein Team KI-fähig machen müsstest, würdest du eher top-down oder bottom-up ansetzen, und warum? Gibt es bei dir bereits eine Skill, die eigentlich Firmenwissen ist, aber nirgendwo als solches festgehalten wurde?

Closing

Der Unterschied zwischen einem enttäuschenden und einem wirklich nützlichen Agenten liegt selten am Modell dahinter. Er liegt daran, ob der Agent onboardet wurde wie ein echter neuer Mitarbeiter: mit eigenen Kontextdateien, die er lesen kann, mit angeschlossenen Werkzeugen, mit denen er handeln kann, und mit Skills, die festhalten, wie du Dinge erledigt haben willst. **Der Kern-Satz bleibt: ein Agent wird nicht durch ein stärkeres Modell besser, sondern durch ein besseres Onboarding.**

Der nächste kleine Schritt: nimm eine einzige Aufgabe, die du in den letzten Wochen mindestens zweimal fast identisch erledigt hast, und sag deinem Agenten am Ende des nächsten Durchlaufs „mach daraus eine Skill“, statt sie ein drittes Mal von Grund auf zu erklären.

Wenn du weiterkommen willst

Diese Unterlage ist ein Werkzeug, kein Ersatz für ein Gespräch. Wenn beim Durcharbeiten etwas hakt, wenn ein Schritt auf deine Lage nicht passt oder du wissen willst, wie das bei euch konkret aussähe: **schreib mir.** Antworte einfach auf die Nachricht, mit der du das hier bekommen hast — oder melde dich über aivantum.com. Ich lese und antworte selbst.

Diese Unterlage wurde mit KI-Unterstützung erstellt und redaktionell geprüft. Weitergabe zum persönlichen Gebrauch erlaubt, gewerbliche Weiterverbreitung nicht.